

PIXIU TECH LLP

Robotics Lab Book

Class 6 | Light, Signal and Simple Automation

Build a traffic signal, a lamp that wakes up in the dark, and a toll gate that opens by itself.

FIVE UNITS Introduction | Arduino IDE | Basic | Intermediate | Final

FIFTEEN SESSIONS One session = one class

THREE PROJECTS Traffic Light - Automatic Night Lamp - Smart Toll Booth

BOARD Arduino Uno

THIS BOOK BELONGS TO _____

SCHOOL / SECTION _____

Contents

Fifteen sessions. One session is one class period.

Unit 1 - Introduction	3
Session 1 - What Is a Robot, and What Is in Your Box	3
Session 2 - Electricity, Circuits and the Breadboard	5
Check Yourself - Unit 1	8
Unit 2 - The Arduino IDE	8
Session 3 - Meet the Board and the Software	8
Session 4 - The Grammar of Arduino Code	11
Session 5 - Three LEDs, One Board	12
Check Yourself - Unit 2	14
Unit 3 - Basic Project: Traffic Light Signal Controller	14
Session 6 - The Problem and the Plan	14
Session 7 - Wiring and Coding the Signal	15
Session 8 - Testing, Tuning and Caution Mode	17
Check Yourself - Unit 3	18
Unit 4 - Intermediate Project: Automatic Night Lamp	18
Session 9 - A Sensor That Feels Light	18
Session 10 - Building the Night Lamp	19
Session 11 - Calibration: Finding Your Own Number	21
Check Yourself - Unit 4	23
Unit 5 - Final Project: Smart Toll Booth	23
Session 12 - Two New Parts and the Project Brief	23
Session 13 - Building and Wiring the Booth	25
Session 14 - The Program	26
Session 15 - Testing and Your Demonstration	27
Check Yourself - Unit 5	29
Extra Challenges and Twenty Revision Questions	30
Words to Know	31
Answer Key	32
My Project Log	34

When you see	It means
Today you will	The three things this class is about. Read it before you start.
Try This	A short activity to do right now, at your desk, with your kit.
Watch Out	A mistake almost everyone makes. Read it before you wire.
Did You Know	A fact worth taking home and telling someone.
Dark box	Arduino code. Type it exactly - every bracket and semicolon matters.
Challenge	For when you finish early and want something harder.

UNIT 1

Introduction

Sessions 1 and 2

Before you write a single line of code you need two things: the names of the parts in front of you, and a clear idea of how electricity moves. This unit has no programming at all - and yet by the end of it you will have made a light glow.

By the end of this unit you will be able to:

- Name every part in your kit and say what it is for.
- Explain the Sense - Think - Act loop with your own example.
- Draw a circuit and explain why it must be a complete loop.
- Use a breadboard correctly and light an LED with no code.

SESSION 1

one class

What Is a Robot, and What Is in Your Box

Today you will

- Learn what really makes a machine a robot.
- Open your kit and learn the name of every part.
- Agree the seven rules of the robotics lab.

When somebody says the word *robot*, most people picture a metal human with glowing eyes. Real robots almost never look like that. A robot is any machine that can **sense** something about the world, **decide** what that means, and then **act** on the decision.

By that definition there are robots all around you already. The automatic hand dryer in a mall senses your hands, decides they are there, and blows air. A washing machine senses that the drum is unbalanced, decides to slow down, and does it. Neither of them looks like a robot. Both of them behave like one.

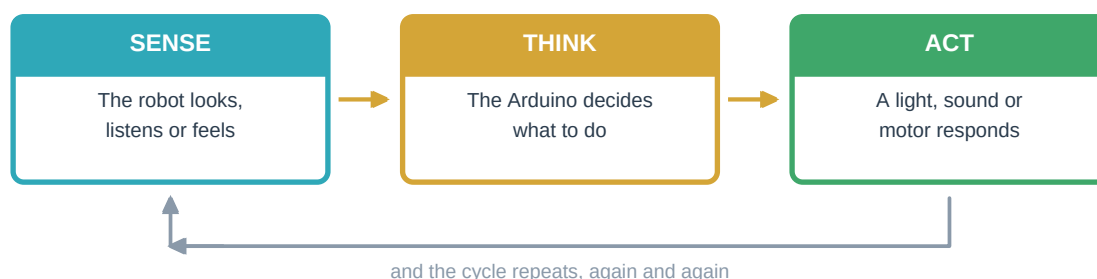


Figure 1.1 - Every robot in the world follows this same three-step loop

Every single thing you build this year follows this loop. In Unit 3 your machine will only act - it will have no senses at all. In Unit 4 it gets its first sense. In Unit 5 it senses, thinks and moves a real object. Keep this diagram in your head; you will come back to it many times.

Try This

Take one minute and look around the classroom. Find three machines that sense something. For each, write down what it senses, what it decides, and what it does. A fan regulator, a mobile phone screen and a fire alarm are good places to start.

1.1 Open the Box

Now open your kit and lay everything out on the desk. Do not plug anything in yet. Your only job in this session is to learn the names. Half of robotics is being able to say exactly what you are holding - it is impossible to ask for help with 'that small blue thing'.

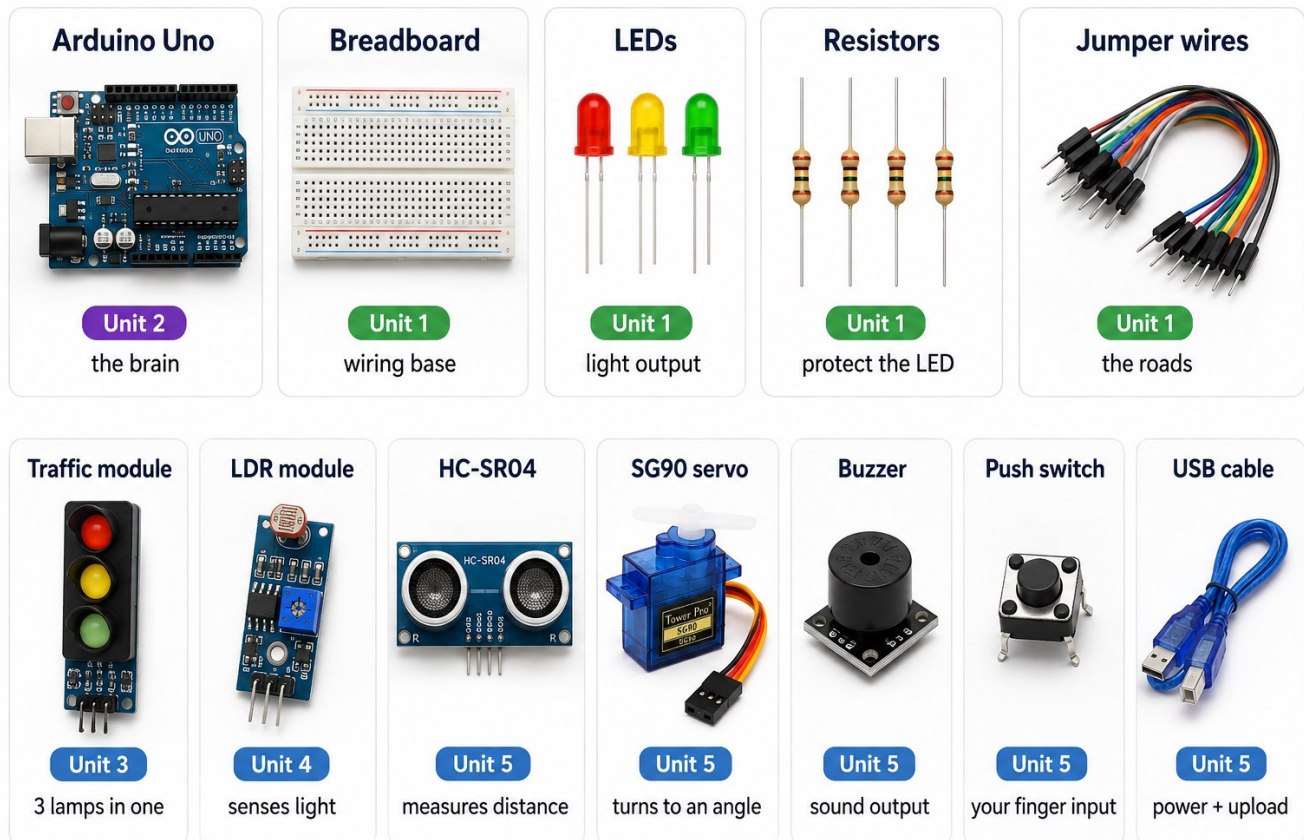


Figure 1.1 - Your kit. Learn these twelve names before the next class.

Part	What it does	First used in
Arduino Uno board	The brain. It remembers your program and follows it forever, even after the power has been off.	Session 3
USB cable	Carries your program from the laptop into the board, and powers the board while you test.	Session 3
Breadboard	A plastic block full of holes that joins parts together without soldering.	Session 2
Jumper wires	The roads between parts. They come as male-to-male, male-to-female and female-to-female.	Session 2
LEDs	Tiny lamps. They glow only one way round.	Session 2
Resistors	Slow the current down so the LED is not destroyed.	Session 2
Traffic light module	Red, yellow and green lamps already built onto one small board.	Session 7
LDR sensor module	Feels how bright the room is and turns it into a number.	Session 9

Part	What it does	First used in
HC-SR04 ultrasonic sensor	Measures how far away an object is, without touching it.	Session 12
SG90 servo motor	Turns to an exact angle and holds it. This will be your barrier arm.	Session 12
Buzzer	Makes sound, so your machine can warn people.	Session 13
Push switch	Lets a human tell the machine something.	Challenge work

1.2 Rules of the Robotics Lab

These seven rules are not decoration. Each one exists because somebody, somewhere, broke a board by ignoring it. Read them once now and follow them for the rest of the year.

- 1 Always unplug the USB cable before you change any wire.
- 2 Check the long leg of every LED and the + side of every module before power goes on.
- 3 Never run a wire straight from 5V to GND. That is a short circuit and it can kill the board.
- 4 Hold the Arduino by its edges. Do not touch the metal pins with wet hands.
- 5 One person wires, one person checks, one person writes the notebook. Then swap roles.
- 6 If anything smells hot or feels hot, unplug it at once and call your trainer.
- 7 Count every component back into the kit box before you leave.

Watch Out

A circuit that does not work is completely normal. It is not failure - it is step one of **debugging**. Professional engineers spend more time finding faults than building. Every fault you track down teaches you something a working circuit never will.

New word	What it means
Robot	A machine that senses, decides and acts.
Component	Any single part in your kit.
Debugging	Hunting down the reason something does not work.

SESSION 2

one class

Electricity, Circuits and the Breadboard

Today you will

- Understand voltage, current and resistance without any maths.
- Learn why a circuit must be a complete loop.
- Light an LED using the breadboard and no code at all.

Electricity is invisible, which makes it hard to imagine. So engineers borrow a picture from something you can see: water in a pipe. The battery is the pump, the wires are the pipes, and the LED is a small fountain at the far end. Once you hold that picture, three words start to make sense.

Word	In the water picture	In electricity
Voltage, measured in volts (V)	How hard the pump pushes	How hard the battery pushes electricity along. Your Arduino pushes at 5V.
Current, measured in amps (A)	How much water flows every second	How much electricity flows every second. Too much current burns parts out.

Word	In the water picture	In electricity
Resistance, measured in ohms	A narrow section of pipe	A part that makes it harder for electricity to flow. That is a resistor's whole job.

Now the single most important rule in this book: **electricity only flows in a complete loop**. It has to leave the battery, travel through every part, and arrive back at the battery. Break the loop anywhere and everything stops instantly. That is not a fault - it is exactly how a switch works. A switch is simply a gap you can open and close on purpose.

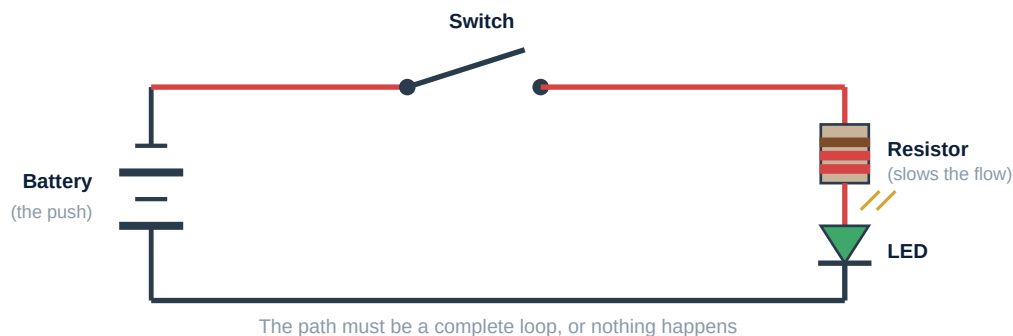
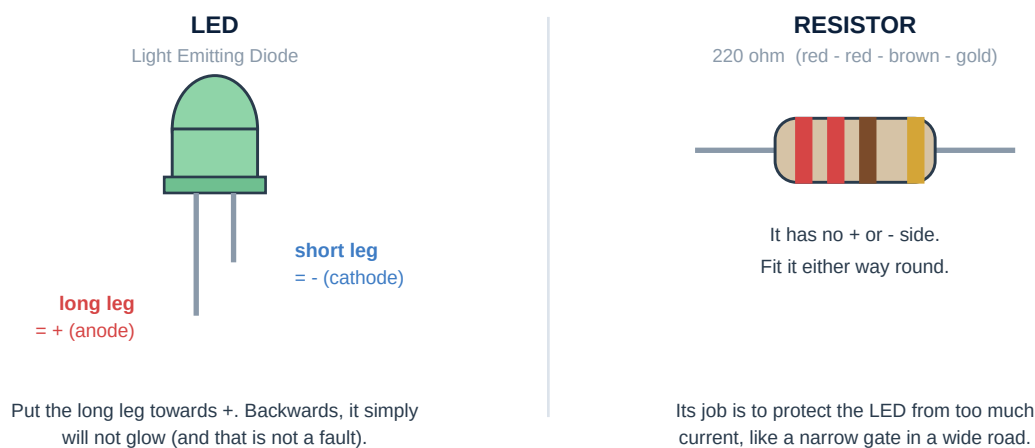


Figure 1.2 - A simple circuit: battery, switch, resistor and LED

2.1 The Two Parts You Will Use Most

An LED and a resistor almost always travel together. The LED gives you light. The resistor keeps the LED alive. Connect an LED to 5V with no resistor and it receives far more current than it can survive - it will glow very brightly for about a second, and then never again.



Watch Out

An LED has a long leg and a short leg. Long leg towards +, short leg towards -. If your LED does not glow, turn it around before you check anything else. Nine times out of ten that is the entire problem.

2.2 Inside the Breadboard

A breadboard looks like a boring block of holes. It is actually full of hidden metal strips. Two holes joined by a hidden strip behave exactly as if you had twisted those two wires together. Knowing which holes are secretly joined is the most useful thing in this whole unit - and the commonest reason a beginner's circuit does nothing at all.

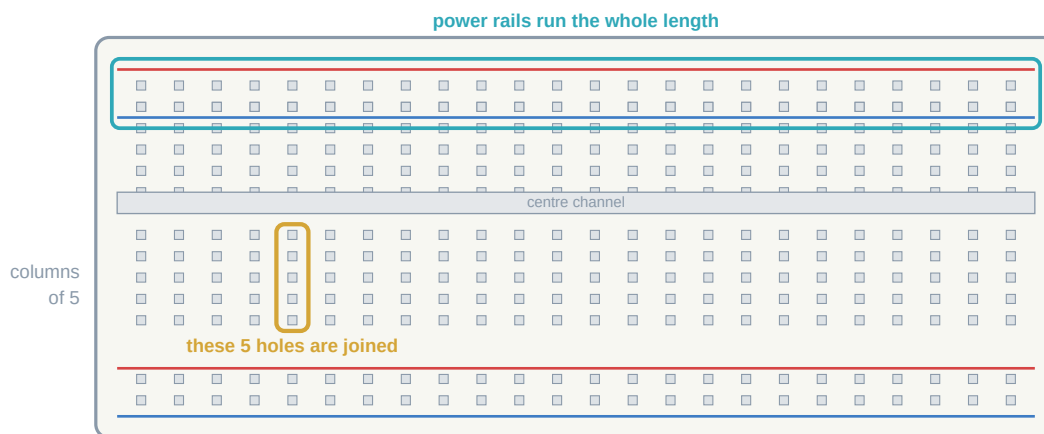


Figure 1.3 - Inside a breadboard: which holes are secretly connected

- The **long side rails**, marked + and -, run the entire length of the board. Use them for power and ground so that every part can reach them.
- In the middle, holes are joined in **columns of five**, up and down only - never across.
- The **centre channel** splits the board into two halves. Nothing crosses it.
- So: two legs in the same column of five are connected. Two legs in different columns are not. That is the whole secret.

HANDS ON**Activity - Light an LED With No Code At All**

Here you will not use the Arduino as a computer. You will use it only as a battery, to prove to yourself that you understand a circuit.

- 1 Push an LED into the breadboard so its two legs sit in **different** columns.
- 2 Push one leg of a 220 ohm resistor into the same column as the LED's **long** leg.
- 3 Run a jumper wire from the resistor's free leg to the **5V** pin on the Arduino.
- 4 Run another jumper wire from the LED's **short** leg column to any **GND** pin.
- 5 Now plug in the USB cable. The LED should glow steadily.
- 6 Pull out the GND wire. The light dies - you have just broken the loop, exactly like a switch.

If it does not glow, check in this order

1. Is the LED the right way round?
2. Are both legs in different columns?
3. Is the resistor really in the same column as the long leg?
4. Is the USB firmly plugged in at both ends?

Did You Know

LED stands for Light Emitting Diode. A diode is a one-way gate for electricity. That is the real reason an LED refuses to work backwards - it is not broken, it is doing its job perfectly.

New word	What it means
Voltage	How hard electricity is pushed. Measured in volts.
Current	How much electricity flows each second. Measured in amps.
Resistor	A part that limits current. Measured in ohms.
Ground (GND)	The return path. Every circuit needs one.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 1**A. Choose the correct answer**

1. A robot must be able to sense, think and _____.
(a) sleep (b) act (c) talk (d) fly
2. Electricity flows only when the circuit is
(a) broken (b) hot (c) a complete loop (d) made of plastic
3. The job of a 220 ohm resistor next to an LED is to
(a) make it brighter (b) protect it (c) change its colour (d) hold it in place
4. In the middle of a breadboard, holes are joined in
(a) columns of five (b) rows of ten (c) no pattern (d) pairs
5. The long leg of an LED must be connected towards
(a) GND (b) the minus side (c) the plus side (d) either side
6. Which of these is measured in ohms?
(a) voltage (b) current (c) resistance (d) time

B. Fill in the blanks

- 1 The battery pushes electricity along. That push is called _____.
- 2 A switch works by _____ the circuit on purpose.
- 3 The two long side rails of a breadboard are used for _____ and ground.
- 4 LED is short for Light _____ Diode.

C. Answer in one or two lines

- 1 Name one machine at home that follows Sense - Think - Act, and explain each step.
- 2 Your LED does not glow. Write the first three things you would check, in order.
- 3 Why can you never join 5V straight to GND with a plain wire?
- 4 Explain in your own words why a resistor is needed next to an LED.

UNIT 2

The Arduino IDE

Sessions 3, 4 and 5

Now the board gets a brain transplant - yours. You will meet the Arduino Uno properly, install the software, and send your very first program down the USB cable into a real piece of hardware.

By the end of this unit you will be able to:

- Point to every important part of the Arduino Uno board.
- Select the right board and port and upload a program successfully.
- Explain what setup() and loop() do, and why the difference matters.
- Write your own code that switches LEDs on and off in a chosen order.

SESSION 3

one class

Meet the Board and the Software

Today you will

- Learn the parts of the Arduino Uno board.
- Install the Arduino IDE and set the board and port.
- Upload the Blink program and watch a real LED obey you.

The Arduino Uno is a small computer with no screen and no keyboard. It cannot run apps. It runs exactly one program - the one you give it - over and over, forever. It keeps that program in memory even after the power is pulled out, and starts running it again the instant power returns. That is why a machine built on an Arduino still works when you unplug it from the laptop and power it from a battery.

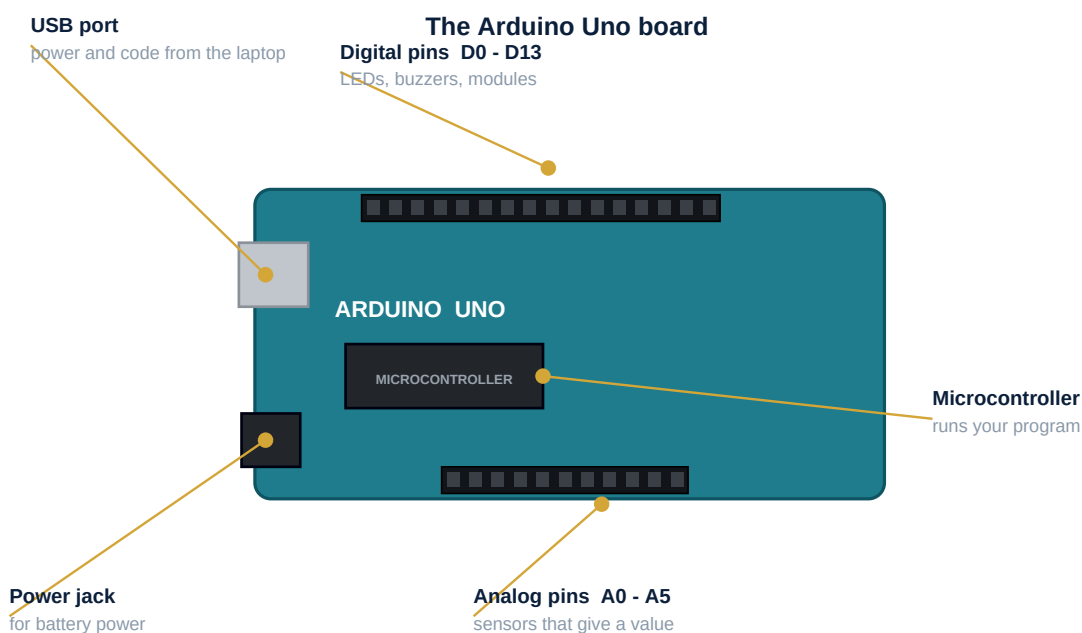


Figure 2.1 - Know your board before you wire anything to it

Part of the board	What you use it for
Digital pins D0 to D13	Anything that is simply ON or OFF: LEDs, buzzers, modules.
Analog pins A0 to A5	Sensors that give a value rather than just on or off.
5V and GND pins	Power out to your components. Every circuit needs both of these.
USB port	Sends your program in from the laptop, and powers the board.
Power jack	For running your finished machine from a battery instead.
Reset button	Restarts your program from the very beginning.
Pins marked with ~	Can fade an output smoothly instead of only switching it.

3.1 The Software: Arduino IDE

IDE stands for Integrated Development Environment, which is a very long name for the window where you type your program. Ignore most of the buttons for now. Two of them matter more than all the rest: **Verify** checks your code for mistakes without sending it anywhere, and **Upload** sends the checked code into the board.

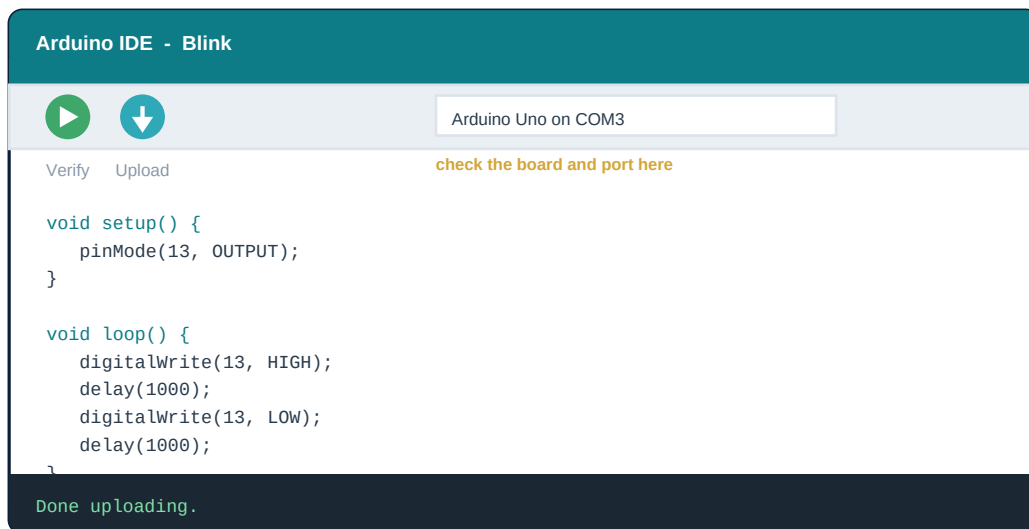


Figure 2.2 - Verify checks your code; Upload sends it to the board

Watch Out

Before you press Upload for the very first time, check two settings. **Tools > Board** must say Arduino Uno, and **Tools > Port** must have a port selected. When an upload fails, this is the first thing to check - every single time, for the rest of your life.

3.2 Your First Program: Blink

Every Arduino learner on the planet starts at exactly the same place. Blink switches an LED on for one second, off for one second, and repeats forever. Build the circuit first, then type the code.

D13 -> 220 ohm resistor -> LED -> GND

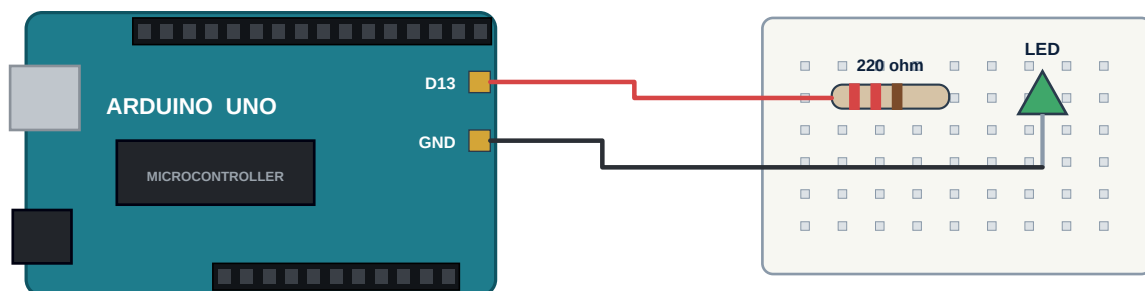


Figure 2.3 - Your first circuit that a program controls

Blink.ino

```
int led = 13;           // the pin our LED is connected to

void setup() {
  pinMode(led, OUTPUT); // this pin will send power out
}

void loop() {
  digitalWrite(led, HIGH); // switch the LED on
  delay(1000);             // wait 1000 milliseconds = 1 second
  digitalWrite(led, LOW);  // switch the LED off
}
```

```
delay(1000);           // wait again
}
```

Press Verify. If the bar at the bottom turns green and says *Done compiling*, your grammar is correct. Now press Upload. The two small lights near the USB socket will flicker as your program travels down the cable. A second later, your LED begins to blink - and it is doing that because of a sentence you wrote.

SESSION 4

one class

The Grammar of Arduino Code

Today you will

- Learn what every word in your program actually means.
- Understand the difference between `setup()` and `loop()`.
- Read an error message and work out what it is telling you.

Arduino code looks frightening for about ten minutes, and then it looks obvious. That is because there are only a handful of words to learn, and each one does exactly what its name suggests. Here is the whole vocabulary you need for Units 1 to 4.

What you type	What it really means
<code>void setup()</code>	Do everything inside these brackets once , the moment the board wakes up.
<code>void loop()</code>	Do everything inside these brackets again and again , forever, until the power is cut.
<code>pinMode(9, OUTPUT)</code>	Pin 9 is going to send power out to something.
<code>pinMode(2, INPUT)</code>	Pin 2 is going to listen to something instead.
<code>digitalWrite(9, HIGH)</code>	Send 5V out of pin 9 - switch it on.
<code>digitalWrite(9, LOW)</code>	Send 0V out of pin 9 - switch it off.
<code>delay(1000)</code>	Freeze here for 1000 milliseconds, then carry on.
<code>int led = 13;</code>	Give the number 13 a name, so the code reads like English.
<code>// like this</code>	A comment. The board ignores it completely; humans read it.
<code>;</code> at the end of a line	A full stop. Almost every instruction needs one.
<code>{ }</code>	Brackets that hold a group of instructions together. They always come in pairs.

The difference between `setup()` and `loop()` is worth one extra minute. Think of `setup()` as getting dressed in the morning - you do it once, and then you get on with your day. `loop()` is the day itself, repeating until you sleep. Telling the board *pin 9 is an output* belongs in `setup()`, because it does not need saying twice. Telling the board *switch pin 9 on* belongs in `loop()`, because you want it to keep happening.

Did You Know

`delay(1000)` means one thousand **milliseconds**, and a millisecond is one thousandth of a second. So `delay(1)` would blink the LED a thousand times a second - far too fast for your eye. It would simply look permanently half-bright.

4.1 Errors You Will Definitely Meet

Error messages look unfriendly, but they are trying to help. The trick is that the error is often reported on the line *after* the real mistake. Learn these five and you will solve most of your own problems without help.

What the screen says	What is actually wrong	How to fix it
expected ';' before ...	You forgot a semicolon	Add ; at the end of the line above

What the screen says	What is actually wrong	How to fix it
'digitalwrite' was not declared	Wrong capital letters	Arduino is fussy: digitalWrite, not digitalwrite
expected '}' at end of input	A missing closing bracket	Count your { and } - they must pair up exactly
Board at COM__ is not available	Wrong port, or a charging-only cable	Re-select Tools > Port, or try a different USB cable
Code uploads but nothing happens	The fault is in the wiring, not the code	Check LED direction, the resistor and the GND wire

Try This

Deliberately break your working Blink program. Delete one semicolon and press Verify. Read the message carefully, then put the semicolon back. Doing this on purpose once means the message will never frighten you again.

SESSION 5

one class

Three LEDs, One Board

Today you will

- Wire three LEDs, each with its own resistor.
- Write a chase light that runs across all three.
- Take on the SOS challenge using only delay and digitalWrite.

One LED proves your code works. Three LEDs prove you can control **order** and **timing** - and order and timing are the entire idea behind your first project. Wire an LED with a 220 ohm resistor to pin D8, another to D9, and a third to D10. All three short legs go to the same GND rail on the breadboard.

ChaseLight.ino

```
int a = 8;
int b = 9;
int c = 10;

void setup() {
  pinMode(a, OUTPUT);
  pinMode(b, OUTPUT);
  pinMode(c, OUTPUT);
}

void loop() {
  digitalWrite(a, HIGH); delay(200); digitalWrite(a, LOW);
  digitalWrite(b, HIGH); delay(200); digitalWrite(b, LOW);
  digitalWrite(c, HIGH); delay(200); digitalWrite(c, LOW);
}
```

Notice that three instructions have been written on one line, separated by semicolons. The board does not care about lines at all - only about semicolons and brackets. Humans care about lines, which is why we usually keep one instruction per line. Here, keeping each LED on its own line makes the pattern easier to read.

Try This

Change every 200 to 60 and upload again. Then change them all to 800. You have just discovered that the same circuit can behave completely differently with no rewiring at all. That is the power of software.

Challenge

Morse code for SOS is three short flashes, three long flashes, three short flashes. Using only one LED, `delay()` and `digitalWrite()`, write the SOS signal. Use 200 ms for short, 600 ms for long, 200 ms for the gap between flashes and 1000 ms before the message repeats.

New word	What it means
Upload	Sending your program from the laptop into the board.
Comment	A note in the code for humans. The board ignores it.
Millisecond	One thousandth of a second.
Compile	Turning your typed words into instructions the chip understands.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 2**A. Choose the correct answer**

- Code inside void setup() runs
(a) forever (b) once at the start (c) only on reset (d) never
- digitalWrite(9, HIGH) means
(a) read pin 9 (b) switch pin 9 on (c) switch pin 9 off (d) fade pin 9
- delay(500) makes the board wait for
(a) 500 seconds (b) 500 minutes (c) half a second (d) 5 seconds
- Before uploading you must always check
(a) the LED colour (b) board and port (c) the room light (d) the battery
- Sensors that give a value, not just ON or OFF, connect to
(a) D13 (b) GND (c) analog pins (d) the USB port
- Almost every instruction in Arduino code ends with
(a) a comma (b) a semicolon (c) a bracket (d) nothing

B. Fill in the blanks

- Code that must repeat forever is written inside void _____.
- A line beginning with // is called a _____.
- pinMode(2, _____) tells the board that pin 2 will listen.
- Pins marked with the ~ symbol can _____ an output smoothly.

C. Answer in one or two lines

- Explain the difference between setup() and loop() in your own words.
- Your code uploads successfully but the LED stays dark. Is the problem in the code or the wiring? How would you find out?
- Why do programmers write int led = 13; instead of typing 13 everywhere?
- Write the one line of code that switches pin 7 off.

UNIT 3

Basic Project: Traffic Light Signal Controller

Sessions 6, 7 and 8

Your first real machine. A traffic signal is nothing more than three lamps switched on in a fixed order with the right waiting time between them. Simple - and yet it keeps millions of people alive every single day.

By the end of this unit you will be able to:

- Explain the order and the purpose of red, yellow and green.
- Draw a flowchart before you write any code.
- Connect a four-pin module to the correct digital pins.
- Test your own machine and improve the timing after watching it run.

SESSION 6

one class

The Problem and the Plan

Today you will

- Study how a real traffic signal behaves.
- Decide the timing for red, green and yellow.
- Draw the flowchart for your machine before touching a wire.

Outside your school gate, students and vehicles share the same stretch of road. Nobody knows whose turn it is, so everybody edges forward together. Your job is to build a signal that repeats a safe cycle entirely by itself, with no human pressing anything.

Before an engineer touches a wire, they draw the plan. A **flowchart** is a picture of what a machine will do, step by step, in the order it will do it. It costs five minutes and saves an hour. Here is the cycle you are going to build.

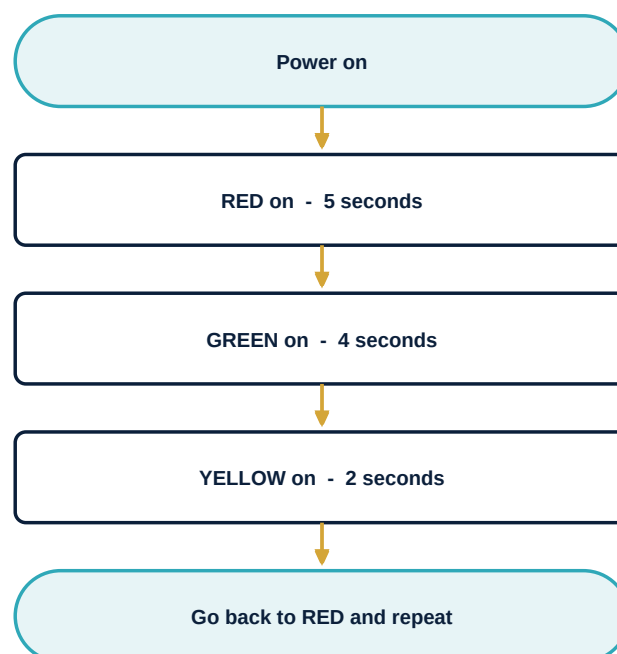


Figure 3.1 - The traffic cycle, drawn before it is coded

Why is yellow needed at all? Because a driver travelling at speed cannot stop instantly. Yellow is a warning that green is ending, giving time to slow down safely. Remove yellow from a real junction and you get collisions. Every part of a machine exists for a reason - always ask what the reason is.

Try This

Before the next class, stand safely near a real signal for two minutes with a watch. Time each lamp. Write the three numbers in your notebook and compare them with the numbers in the flowchart above. Are yours different? Why might that be?

SESSION 7

one class

Wiring and Coding the Signal

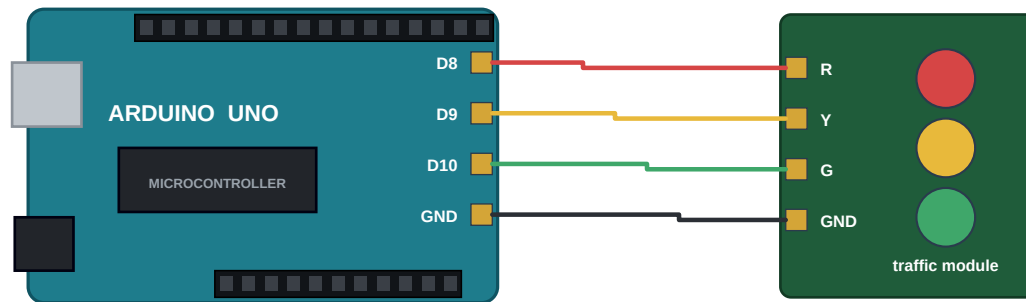
Today you will

- Connect the traffic light module to the correct pins.
- Type, verify and upload the traffic light program.

- See your flowchart running as real hardware.

The traffic light module already has the three lamps *and* their resistors built onto it. That is why you need only four wires instead of eight: one for each lamp, plus one shared ground. This is exactly why modules exist - somebody has done the fiddly part for you.

Traffic light module -> Arduino Uno



Only four wires. Match your jumper colours to the lamps - it saves hours of debugging.

Figure 3.1 - Wiring the traffic light module

Module pin	Arduino pin	Why
R	D8	Red lamp
Y	D9	Yellow lamp
G	D10	Green lamp
GND	GND	The return path. Without this, nothing works at all.

Watch Out

GND is not optional and it is not decoration. Electricity must have a way back to the board or the loop is never completed. Roughly nine out of ten 'my module is dead' problems turn out to be a missing GND wire.

TrafficLight.ino

```
int red    = 8;
int yellow = 9;
int green  = 10;

void setup() {
  pinMode(red, OUTPUT);
  pinMode(yellow, OUTPUT);
  pinMode(green, OUTPUT);
}

void loop() {
  digitalWrite(red, HIGH);    // STOP
  delay(5000);
  digitalWrite(red, LOW);

  digitalWrite(green, HIGH);  // GO
  delay(4000);
  digitalWrite(green, LOW);

  digitalWrite(yellow, HIGH); // GET READY
  delay(2000);
  digitalWrite(yellow, LOW);
}
```


Read the code and the flowchart side by side. They say exactly the same thing - one in pictures for you, one in words the board understands. Notice also that each lamp is switched off before the next one comes on. Delete one of those LOW lines and see what happens; it is a mistake worth making once.

SESSION 8

one class

Testing, Tuning and Caution Mode

Today you will

- Test your signal the way an engineer would.
- Change the timings and justify your choice.
- Add a night-time caution mode as a challenge.

A machine that worked once is not a machine that works. Engineers test deliberately, write down what they see, and only then say the job is done. Work through this table with your group and tick each row honestly.

Test	What should happen	Tick
Power on and watch three full cycles	The same order every time, nothing skipped	■
Count the seconds with a phone timer	Matches the delay values you wrote	■
Wiggle the jumper wires gently	Lamps stay steady, nothing flickers	■
Pull out the GND wire	Everything stops - proof that it matters	■
Ask a friend to watch one cycle	They can correctly predict the next lamp	■
Leave it running for five minutes	No lamp gets warm, nothing resets by itself	■

Now tune it. Watch your own signal for a while and ask whether the timings feel right for a school gate, where people walk rather than drive. Should red be longer? Should yellow be shorter? Change the numbers, upload again, and write down **why** you chose what you chose. An engineer who cannot explain their numbers has only been guessing.

Challenge

Real signals switch to **caution mode** late at night: the yellow lamp alone blinks on and off, once per second, and the other lamps stay dark. Add a caution mode to your code. Then think about how you would let the machine choose between normal mode and caution mode by itself - you will be able to solve that properly after Unit 4.

Did You Know

The order red - green - yellow is not the same everywhere in the world. In several countries the signal shows red and yellow together just before green, warning drivers to get ready. Your code could do that with a single extra digitalWrite.

New word	What it means
Flowchart	A picture of what a machine will do, step by step.
Module	A small ready-made board with parts already fitted on it.
Cycle	One complete run through a repeating sequence.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 3**A. Choose the correct answer**

- The traffic module needs only four wires because
(a) it is small (b) resistors are built in (c) it has no lamps (d) it uses no power
- If the GND wire is removed, the module will
(a) get brighter (b) not work at all (c) change colour (d) run faster
- delay(5000) in the red section means red stays on for
(a) 5 milliseconds (b) 50 seconds (c) 5 seconds (d) 5 minutes
- A flowchart should be drawn
(a) after coding (b) before coding (c) instead of coding (d) never
- To make green last longer you would change
(a) the wire colour (b) the delay value after green (c) pinMode (d) the GND pin
- Yellow exists on a real signal so that drivers can
(a) speed up (b) slow down safely (c) see better (d) turn left

B. Fill in the blanks

- The three lamps are connected to pins _____, _____ and _____.
- Everything inside void loop() repeats _____.
- A flowchart shows the same idea as code, but in _____.
- Late at night a real signal may switch to _____ mode.

C. Answer in one or two lines

- Why is the yellow lamp needed? What would go wrong on a real road without it?
- Write down the delay values you chose and explain why you chose those numbers.
- Describe one change you made after testing, and what made you change it.
- A classmate's module does nothing at all. Name the two wires you would check first.

UNIT 4

Intermediate Project: Automatic Night Lamp

Sessions 9, 10 and 11

Until now your machine has been blind. It followed orders and nothing more. In this unit it gains its first sense, and for the very first time it makes a decision of its own.

By the end of this unit you will be able to:

- Explain how an LDR turns light into a number.
- Read a live sensor value on the Serial Monitor.
- Choose a threshold from real measurements instead of guessing.
- Write an if - else decision that links an input to an output.

SESSION 9

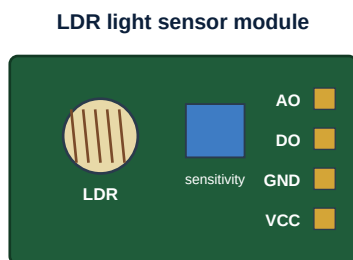
one class

A Sensor That Feels Light

Today you will

- Learn how an LDR works and what its numbers mean.
- Open the Serial Monitor and watch a live sensor value.
- Record the reading in bright light and in darkness.

An LDR - a Light Dependent Resistor - is a small disc with a wiggly line printed across it. In bright light, electricity passes through that wiggly line easily. In darkness, it resists. The Arduino cannot see brightness the way you can. It can only read that resistance as a **number between 0 and 1023**, and it is up to you to decide what those numbers mean.

**What the numbers mean****Bright light**

resistance drops
-> small number on A0

Darkness

resistance rises
-> big number on A0

The Arduino cannot 'see'. It only reads a number from 0 to 1023, and you decide which number counts as dark.

Figure 4.1 - The LDR module and the numbers it gives you

This is one of the biggest ideas in the whole book, so it is worth saying slowly. A sensor does not tell a machine *it is dark*. A sensor gives a number. A human decides which numbers count as dark. That decision is called setting a **threshold**, and choosing it well is the difference between a lamp that works and a lamp that flickers all evening.

Try This

Wire the LDR module: VCC to 5V, GND to GND, AO to A0. Then upload a program with only these two lines inside loop(): `Serial.println(analogRead(A0));` and `delay(200);` - and remember `Serial.begin(9600);` inside setup(). Open Tools > Serial Monitor and cover the sensor with your hand. Write down the number in a bright room and the number under your palm. Those two numbers decide everything that follows.

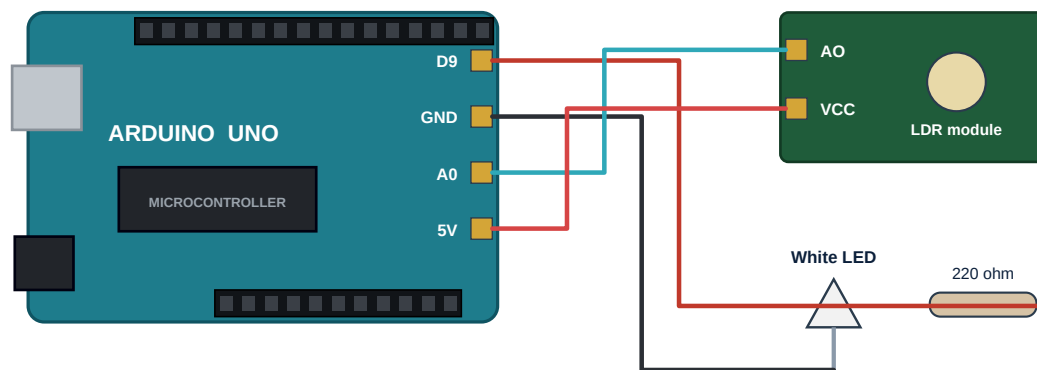
SESSION 10

one class

Building the Night Lamp**Today you will**

- Wire the LDR module and the white LED together.
- Understand the if - else decision.
- Upload the night lamp program and watch it decide.

Automatic night lamp - connections



Sensor goes to the analog side (A0). The lamp goes to the digital side (D9).

Figure 4.2 - Input on one side, output on the other

Component pin	Arduino pin	Why
LDR module VCC	5V	Power for the sensor
LDR module GND	GND	Shared return path
LDR module AO	A0	The light level, as a number
White LED long leg	D9 through a 220 ohm resistor	The street lamp
White LED short leg	GND	Return path

Now for the new idea. Until this moment your programs have only given orders. This one asks a question first. In plain English: *if it is dark, switch the lamp on; otherwise switch it off*. In Arduino that is an **if - else**, and in a flowchart a question is drawn as a diamond rather than a box.

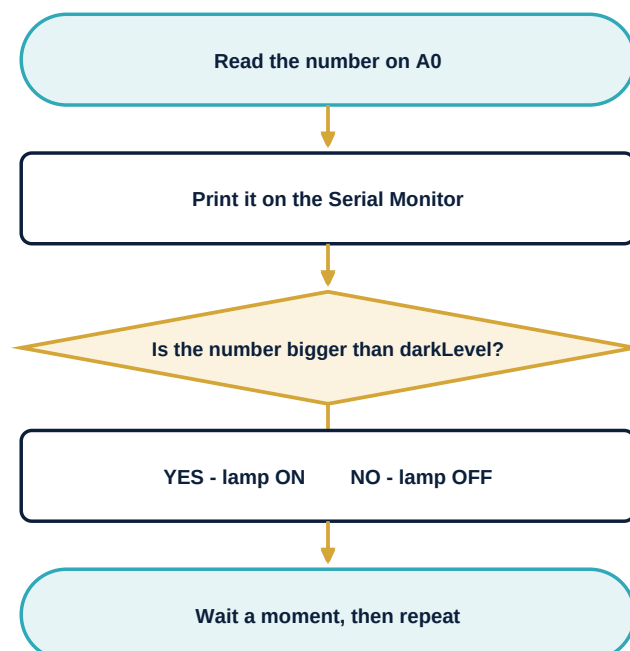


Figure 4.1 - A decision is drawn as a diamond

```

NightLamp.ino
int ldrPin    = A0;
int lampPin   = 9;
  
```

```

int darkLevel = 600; // change this after your own testing

void setup() {
  pinMode(lampPin, OUTPUT);
  Serial.begin(9600); // start talking to the laptop
}

void loop() {
  int light = analogRead(ldrPin); // a number from 0 to 1023
  Serial.println(light); // show it on the Serial Monitor

  if (light > darkLevel) {
    digitalWrite(lampPin, HIGH); // it is dark - lamp ON
  } else {
    digitalWrite(lampPin, LOW); // bright enough - lamp OFF
  }

  delay(200);
}

```

Watch Out

600 is only a starting guess, and our classroom is not your classroom. If your lamp behaves backwards - on in daylight and off in the dark - do not rewire anything. Simply change $>$ to $<$. Some LDR modules count upwards in darkness and others count downwards.

SESSION 11

one class

Calibration: Finding Your Own Number**Today you will**

- Measure real readings in your own classroom.
- Choose and justify your threshold value.
- Improve the lamp so it does not flicker at dusk.

Calibration means tuning a machine to the real world it has to live in. A thermometer is calibrated against boiling water. A weighing scale is calibrated against a known weight. Your night lamp is calibrated against your own classroom, at the time of day you actually use it.

Condition	Number on Serial Monitor	Lamp should be
Bright classroom, tube lights on	_____	OFF
Lights off, curtains drawn	_____	ON
Hand fully covering the sensor	_____	ON
Sensor near the window at 5 pm	_____	your decision
The darkLevel you chose	_____	roughly halfway between

There is one more problem to solve. At dusk the light level sits almost exactly on your threshold, and tiny changes make the lamp flicker on and off repeatedly. Real engineers fix this by making the machine slightly stubborn: once the lamp is on, it needs the light to rise well above the threshold before it will switch off again. Try adding a `delay(1000)` after each switch and see how much steadier the lamp becomes.

Challenge

Real street lamps do not simply jump from off to full brightness. Add a second LED so the lamp glows dim at dusk and bright in full darkness, using `analogWrite(pin, value)` where value runs from 0 to 255. Hint: only the pins marked with ~ can do this.

New word	What it means
Sensor	A part that turns something physical into a number.
Analog	A value anywhere in a range, not just on or off.
Threshold	The number at which a decision changes from one answer to the other.
Calibration	Adjusting a machine to suit the real place it works in.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 4**A. Choose the correct answer**

1. `analogRead()` gives a number between
(a) 0 and 1 (b) 0 and 255 (c) 0 and 1023 (d) 0 and 5
2. The LDR module's AO pin must be connected to
(a) D13 (b) A0 (c) GND only (d) the USB port
3. `Serial.begin(9600)` is needed so that
(a) the LED glows (b) the board can talk to the laptop (c) the sensor works (d) code compiles
4. A threshold value should be chosen
(a) by guessing (b) from real readings (c) from the textbook (d) at random
5. `if - else` lets a program
(a) repeat forever (b) make a choice (c) wait (d) measure time
6. A sensor tells the machine
(a) that it is dark (b) a number (c) what to do (d) the time

B. Fill in the blanks

- 1 LDR stands for Light _____ Resistor.
- 2 The Serial _____ is the window where you watch live sensor numbers.
- 3 Tuning a machine to its real surroundings is called _____.
- 4 A question in a flowchart is drawn as a _____.

C. Answer in one or two lines

- 1 Write the two numbers you measured, bright and dark, and the `darkLevel` you chose between them.
- 2 Why is it better to print the sensor value before writing the `if - else`?
- 3 Give one real place where an automatic night lamp would save electricity.
- 4 Your lamp flickers rapidly at dusk. Explain why, and one way to reduce it.

UNIT 5

Final Project: Smart Toll Booth

Sessions 12, 13, 14 and 15

Everything you have learnt now comes together in one machine: a sensor that measures distance, a motor that moves a real barrier, a buzzer, two lamps, and a counter that remembers how many vehicles have passed. This is the project you will demonstrate.

By the end of this unit you will be able to:

- Explain how an ultrasonic sensor measures distance using an echo.
- Control a servo motor to move a physical part to an exact angle.
- Combine one sensor and several outputs in a single program.
- Build, test, present and defend your own working machine.

SESSION 12

one class

Two New Parts and the Project Brief

Today you will

- Learn how the HC-SR04 and the SG90 servo work.
- Understand what your finished toll booth must do.
- Agree the four team roles and start the cardboard build.

The HC-SR04 works exactly like a bat. It sends out a click of sound far too high for human ears, then listens for the echo and times how long it takes to come back. Sound travels about 34 centimetres every millisecond, so the time tells you the distance. The SG90 servo is a different kind of motor: instead of spinning endlessly, you tell it an angle and it turns there and holds position until you say otherwise.

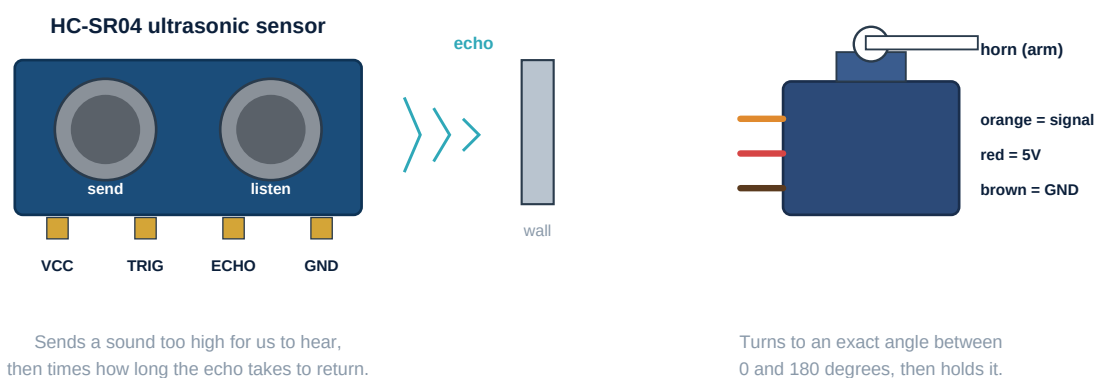
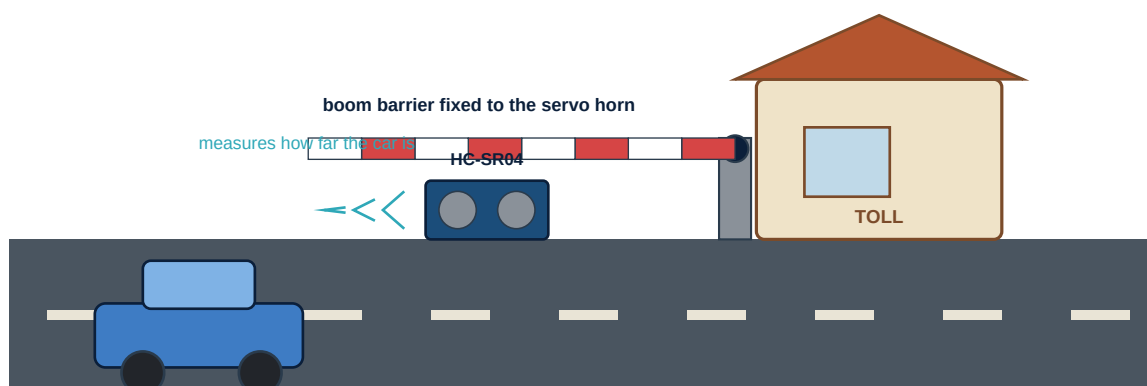


Figure 5.1 - The two new parts in your final project

Did You Know

The echo travels to the object **and back again**, so it covers double the distance. That is exactly why the formula divides by two. Bats, dolphins and submarines all use this same trick, and it has a name: **echolocation**.

12.1 What You Are Building



Car comes close -> barrier lifts -> car passes -> barrier closes -> count + 1

Figure 5.2 - What you are building: an automatic toll gate

Four jobs have to happen at once in this project, so decide now who is doing what. Swap roles between sessions so that everybody does everything at least once.

Role	Responsible for	Name
Builder	Cardboard booth, barrier arm, mounting the sensor firmly	_____
Wiring engineer	Every connection, and checking each one twice	_____
Programmer	Typing, uploading and changing values during testing	_____
Recorder	The notebook: test results, faults, and what fixed them	_____

SESSION 13

one class

Building and Wiring the Booth

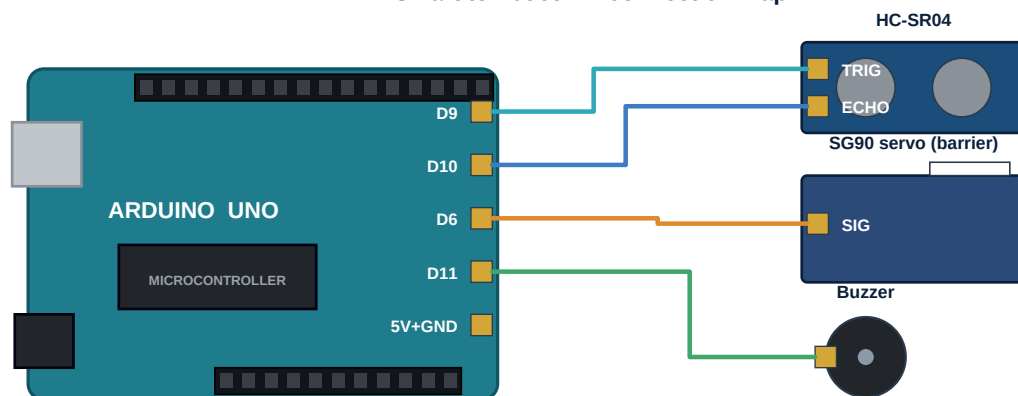
Today you will

- Build the cardboard booth and fix the barrier to the servo horn.
- Mount the ultrasonic sensor facing along the road.
- Complete every connection and check them before power on.

Build the physical model first and the wiring second. A barrier that is too heavy for the servo, or a sensor pointed at your own booth wall, will look like a code problem for an hour before you realise it was never a code problem at all.

- Keep the barrier arm light - a strip of cardboard is plenty. A heavy arm makes the servo tremble.
- Fix the servo firmly. If the servo body moves instead of the arm, nothing works properly.
- Point the HC-SR04 **along** the road, not across it, or it will see the booth instead of the car.
- Mount the sensor at roughly the height of your toy car, not above it.

Smart toll booth - connection map



Every module also needs 5V and GND. Share them through the breadboard power rails.

Figure 5.3 - One sensor in, three things out

Component	Arduino pin
HC-SR04 TRIG	D9
HC-SR04 ECHO	D10
Servo signal (orange wire)	D6
Buzzer +	D11
PASS lamp (blue LED + 220 ohm)	D12
STOP lamp (orange LED + 220 ohm)	D13
All VCC and red wires	5V
All GND, brown and black wires	GND

Watch Out

The servo, the sensor, the buzzer and both lamps all need 5V and GND. The Arduino has only a few of each, so run one wire from 5V and one from GND to the breadboard power rails, and take everything else from there.

SESSION 14

one class

The Program

Today you will

- Read and type the longest program in this book.
- Learn how to write your own instruction.
- Upload, and get the barrier moving for the first time.

This program is longer than anything you have written so far, so read it in pieces rather than all at once. There is one genuinely new idea in it: we have written **our own instruction**, called `readDistance()`. Everything needed to measure distance is tucked inside it, so the main loop stays short and readable. Professional programmers do this constantly - it is how large programs stay understandable.

SmartTollBooth.ino

```
#include <Servo.h>           // borrow ready-made servo instructions

Servo barrier;
int trig = 9;
int echo = 10;
int buzzer = 11;
int passLed = 12;
int stopLed = 13;
int count = 0;              // vehicles counted so far

void setup() {
  barrier.attach(6);
  pinMode(trig, OUTPUT);
  pinMode(echo, INPUT);
  pinMode(buzzer, OUTPUT);
  pinMode(passLed, OUTPUT);
  pinMode(stopLed, OUTPUT);
  barrier.write(0);          // start with the barrier closed
  digitalWrite(stopLed, HIGH);
  Serial.begin(9600);
}

int readDistance() {         // our own instruction
  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);
  long t = pulseIn(echo, HIGH);
  return t * 0.034 / 2;      // time turned into centimetres
}

void loop() {
  int d = readDistance();

  if (d > 0 && d < 15) {     // a vehicle is close
    digitalWrite(stopLed, LOW);
    digitalWrite(passLed, HIGH);
    digitalWrite(buzzer, HIGH);
  }
}
```

```

barrier.write(90);      // lift the arm
delay(3000);            // hold it open
digitalWrite(buzzer, LOW);
barrier.write(0);       // close it again
digitalWrite(passLed, LOW);
digitalWrite(stopLed, HIGH);

count = count + 1;
Serial.print("Vehicles today: ");
Serial.println(count);
delay(1000);            // stops one car being counted twice
}
delay(100);
}

```

Two details are worth pausing on. The symbol `&&` means *and*, so the barrier lifts only when the distance is greater than 0 **and** less than 15. And `count = count + 1;` means *take the number in count, add one, and put it back* - which is how a machine remembers something across time.

SESSION 15

one class

Testing and Your Demonstration

Today you will

- Find and fix the faults in your own machine.
- Pass every acceptance test.
- Present your project in three minutes.

Almost nothing works perfectly the first time, and that is expected. Here are the five faults that appear most often, and what actually causes each one.

Problem you may see	Most likely cause	What to do
Barrier trembles and buzzes	The servo is pulling more current than the port can give	Use a laptop USB port, not a weak phone charger
The same car is counted many times	The loop runs far faster than the car moves	Increase the delay after counting
Distance always reads 0	TRIG and ECHO are swapped	Swap D9 and D10, then test again
Barrier opens with nothing there	The sensor is aimed at the booth wall	Point it along the road, not across it
Nothing happens at all	Missing shared GND	One GND wire must reach every module

Acceptance test	Result	Tick
Barrier lifts within 1 second of a car at 15 cm	_____	■
Barrier stays open for about 3 seconds	_____	■
Barrier closes fully every single time	_____	■
The counter increases exactly once per vehicle	_____	■
Ten cars in a row with no failure	_____	■

15.1 Your Three-Minute Demonstration

Plan your three minutes. Practise it once before you stand up.

- 1 The problem** - one sentence on what your machine solves.

- 2 **How it senses** - hold up the HC-SR04 and explain the echo in your own words.
- 3 **How it decides** - show the line with the number 15 in it and say what it means.
- 4 **How it acts** - run it live with a toy car. Let the barrier speak for you.
- 5 **What went wrong** - name one fault you found and fixed. Judges like this part most.
- 6 **What next** - one honest idea for improving it.

Challenge

Add the push switch as a 'toll paid' button so the barrier lifts only after payment is confirmed. Then print the money collected next to the vehicle count, assuming 20 rupees per vehicle.

New word	What it means
Ultrasonic	Sound too high for human ears to hear.
Echolocation	Finding distance by timing an echo.
Servo	A motor that turns to an exact angle and holds it.
Function	An instruction you write yourself, such as readDistance().

CHECK WHAT YOU LEARNT

Check Yourself - Unit 5**A. Choose the correct answer**

1. The HC-SR04 measures distance by
(a) seeing light (b) timing an echo (c) touching the object (d) weighing it
2. The distance formula divides by 2 because
(a) it is easier (b) sound travels there and back (c) the sensor is slow (d) of the resistor
3. `barrier.write(90)` tells the servo to
(a) spin forever (b) turn to 90 degrees (c) stop working (d) wait 90 seconds
4. `readDistance()` in this program is
(a) a built-in word (b) an instruction we wrote (c) a mistake (d) a comment
5. The delay after counting a vehicle prevents
(a) overheating (b) counting the same car twice (c) servo damage (d) code errors
6. The symbol `&&` in an if statement means
(a) or (b) and (c) not (d) equals

B. Fill in the blanks

- 1 A servo motor turns to an exact _____ and holds it there.
- 2 The signal wire of an SG90 servo is usually _____ in colour.
- 3 Sound travels roughly _____ centimetres every millisecond.
- 4 `count = count + 1;` is how a machine _____ something across time.

C. Answer in one or two lines

- 1 Explain echolocation to someone in Class 4, in two sentences.
- 2 Your barrier opens but never closes. Name two things you would check.
- 3 Name one useful thing a real toll company could do with the vehicle count.
- 4 Which of the three projects taught you the most, and why?

Extra Challenges

Finished early? Pick any of these. None of them need components you do not already have.

#	Challenge	Uses
1	Make an LED fade smoothly instead of blinking	analogWrite, ~ pins
2	Build a two-road junction: when one is green, the other is red	6 LEDs
3	A lamp that blinks faster the darker it gets	LDR + delay
4	A reversing alarm: the buzzer beeps faster as your hand comes closer	HC-SR04 + buzzer
5	A servo that follows your hand: closer hand, bigger angle	HC-SR04 + servo
6	Count how many times a hand passes the sensor in one minute	HC-SR04 + Serial
7	A doorbell that plays a two-tone chime	switch + buzzer
8	A night lamp that also beeps when it becomes very dark	LDR + buzzer
9	A traffic signal with a night caution mode that blinks only yellow	traffic module
10	A toll gate that shows the money collected, not just the count	Serial + maths

MIXED REVISION

Twenty Questions to Test Yourself

- 1 What are the three steps every robot follows?
- 2 Why must a circuit be a complete loop?
- 3 Which leg of an LED goes towards the plus side?
- 4 What happens if you connect an LED with no resistor?
- 5 How many holes are joined together in one breadboard column?
- 6 What is the difference between a digital pin and an analog pin?
- 7 What does void setup() do that void loop() does not?
- 8 Write the line of code that switches pin 7 on.
- 9 How many milliseconds are there in three seconds?
- 10 What are the first two settings to check before uploading?
- 11 Why does the traffic module need only four wires?
- 12 What is the largest number analogRead() can give?
- 13 What does calibration mean?
- 14 Why do we print sensor values before writing an if - else?
- 15 Explain what a threshold is, in one sentence.
- 16 How does the HC-SR04 know how far away something is?
- 17 Why does the distance formula divide by two?
- 18 What is the difference between a servo motor and an ordinary DC motor?
- 19 Why do programmers write their own instructions like readDistance()?
- 20 Name one thing you would add to your toll booth with one more session.

Words to Know

Word	Meaning
Actuator	Any part that moves or acts: a motor, a buzzer, a lamp.
Analog	A value that can be anything in a range, not just on or off.
Breadboard	A block of holes for joining parts without soldering.
Calibration	Adjusting a machine to suit the real place it works in.
Compile	Turning typed words into instructions the chip can follow.
Current	How much electricity flows every second.
Debugging	Hunting down the reason something does not work.
Digital	A value that is only ON or OFF, HIGH or LOW.
Echolocation	Finding distance by timing an echo. Bats do it; so does the HC-SR04.
Flowchart	A picture of what a machine will do, step by step.
Function	An instruction you write yourself and then reuse.
Ground (GND)	The return path. Every circuit needs one.
IDE	The software window where you write and upload your program.
LDR	Light Dependent Resistor - a sensor that feels brightness.
LED	Light Emitting Diode - a lamp that works only one way round.
Loop	Code that repeats again and again.
Millisecond	One thousandth of a second.
Module	A small ready-made board with components already fitted.
Resistor	A part that limits current, measured in ohms.
Sensor	A part that turns something physical into a number.
Servo	A motor that turns to an exact angle and holds it.
Threshold	The number where a decision changes from one answer to the other.
Upload	Sending your program from the laptop into the board.
Voltage	How hard electricity is being pushed, measured in volts.

Answer Key

Section A only. Sections B and C are for you and your trainer to discuss - there is often more than one good answer.

Check Yourself - Unit 1

Q	Answer	The correct option
1	(b)	act
2	(c)	a complete loop
3	(b)	protect it
4	(a)	columns of five
5	(c)	the plus side
6	(c)	resistance

Check Yourself - Unit 2

Q	Answer	The correct option
1	(b)	once at the start
2	(b)	switch pin 9 on
3	(c)	half a second
4	(b)	board and port
5	(c)	analog pins
6	(b)	a semicolon

Check Yourself - Unit 3

Q	Answer	The correct option
1	(b)	resistors are built in
2	(b)	not work at all
3	(c)	5 seconds
4	(b)	before coding
5	(b)	the delay value after green
6	(b)	slow down safely

Check Yourself - Unit 4

Q	Answer	The correct option
1	(c)	0 and 1023
2	(b)	A0
3	(b)	the board can talk to the laptop
4	(b)	from real readings
5	(b)	make a choice
6	(b)	a number

Check Yourself - Unit 5

Q	Answer	The correct option
1	(b)	timing an echo
2	(b)	sound travels there and back
3	(b)	turn to 90 degrees
4	(b)	an instruction we wrote
5	(b)	counting the same car twice
6	(b)	and

My Project Log

Fill one row after every class. This is your evidence, and it counts towards your assessment.

Session	What I built or learnt	A problem I hit	How I fixed it
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			

A last word

Nothing in this book worked the first time when we built it either. The wires came loose, the servo trembled, and the counter once counted a single car four times. Every one of those faults taught us something. Keep the notebook, keep asking why, and next year you will be building things this book cannot teach you.